

opened and parsed your bootchart-20251015-0643.svg and did a hands-on pass through the chart so you get a real, in-depth breakdown (I also displayed the chart and a parsed table of chunks for you to inspect). Below is a complete, practical walkthrough: what each part means, how to read timings, what this particular chart reveals, and concrete, prioritized optimizations you can try on the device (BeagleBone / Yocto style) to speed boot.

Quick summary of what I saw (from the SVG)

- The chart uses a small set of lanes:
 1. Processes (this holds thousands of small task rectangles — the actual processes/commands that ran).
 2. CPU[overall] utilization
 3. IO utilization - read
 4. CPU[overall] wait
 5. IO utilization - write
- The parser found ~8,200 process chunks inside the “Processes” lane (many short-lived tasks or many small rectangles for process activity). That makes the Processes lane dense — typical when many services or udev events execute.
- I could not automatically recover a reliable px→seconds scale from the SVG (the chart SVG I parsed either lacks explicit numeric x-tick texts that the parser could use, or the tick text placement was nonstandard). That means I can rank and compare activities precisely (which tasks use the most horizontal space in the chart), but I can't give absolute wall-clock seconds for every chunk without a known total-time anchor or visible numeric ticks.

→ If you want exact seconds for each item, either (A) tell me the total boot time from systemd-analyze (e.g. systemd-analyze shows a number like 12.345s) or (B) re-export the chart with visible time ticks (or give me the svg again after ensuring the x-axis numeric ticks are embedded). With that, I can convert px→s and list the top offenders with real seconds.

How to read this bootchart (detailed)

1. Horizontal axis = time. Left = kernel start / boot begin, right = later time. Each rectangle's horizontal position = start time; width = duration.
 - On a correctly labeled bootchart you will see numeric tick labels (e.g., 0.0 s, 1.0 s, 2.0 s, ...) — that's the px→seconds mapping.
2. Lanes / swimlanes:
 - Processes lane: each rectangle is a portion of CPU time that a particular process used (or a process on one CPU core). Text labels near rectangles identify the process. Dense lanes mean many short tasks ran.

- CPU utilization lane(s): show CPU busy/wait across time (stacked bars or area charts). Peaks indicate heavy CPU usage.
 - IO read / write lanes: show I/O bursts. If IO bars line up with service rectangles, that service is I/O-heavy (waiting on disk or device).
 - CPU wait: indicates time CPU spent waiting, usually for I/O (a big signal that boot is I/O bound).
3. Colors & stacking: commonly colors indicate active running, sleeping, blocked, etc. (exact palette depends on chart generator). Look for:
- Long thin rectangles in Processes → quick-fire tasks (like many udev rules).
 - Wide rectangles → long-running services (e.g., an init script, fsck, or a network timeout).
 - IO spikes aligned with long waits → I/O bound service causing stall.
4. Interpreting causality:
- If a service rectangle begins, then CPU wait / IO read spikes appear, then the next service starts late — that long IO spike (and high CPU wait) is likely delaying the next stage.
 - If many tiny rectangles appear at the same time, udev is enumerating devices — that can be optimized by reducing udev rules / modules.

What this specific chart suggests (high level conclusions)

- Dense “Processes” lane with thousands of chunks means many short-lived tasks run during boot. That’s a sign of:
 - Many small init scripts or systemd units with short life.
 - Large number of udev events/hotplug handling (typical on boards with many kernel modules or device overlays).
- Significant CPU wait & IO read/write lanes — indicates the system spends a non-trivial amount waiting on I/O (e.g., storage, slow initramfs tasks, or heavy file operations). That’s consistent with:
 - Filesystem checks (fsck), large module loads, or slow storage access (SD card).
- Because I could not derive absolute seconds from the SVG automatically, I ranked by pixel width (relative ordering). The chart shows the Processes lane dominates the horizontal space (so boot time is governed by many process activities + IO).

Immediate, high-impact checks (run these on the target — short commands)

Run these on the board and then share the outputs if you want a next-level, second-accurate analysis:

1. `systemd-analyze`
 - Shows total kernel + initrd + userspace time. Example: `systemd-analyze` → Startup finished in 4.345s (kernel) + 2.123s (userspace) = 6.468s.
2. `systemd-analyze blame`
 - Lists services sorted by time taken. Use this to find the slowest units.
3. `systemd-analyze critical-chain`
 - Shows units on the critical path (those that block other services).
4. `journalctl -b` (or `journalctl -b -u <service>`)
 - Inspect logs for timeouts or long waits.
5. `ls /etc/systemd/system/*.wants/` and `systemctl list-unit-files --state=enabled`
 - See which enabled services you can disable/mask.
6. For udev-heavy devices: `udevadm monitor --property` while booting (or look at `/var/log/udev` if present).

If you run those and paste outputs here I'll map them back to the chart and give exact second measurements for the top offenders.

Concrete & prioritized optimizations (start with the highest impact)

1) Short circuit slow services (fast wins)

- `systemctl disable <service>` for services not needed at boot (e.g., bluetooth, ModemManager, wpasupplicant on a headless device).
- If a service is needed later but not at boot: `systemctl disable` + add a user-level timer or socket activation to start it later.

Commands:

list slow services

`systemd-analyze blame | head -n 20`

disable a service

`sudo systemctl disable --now some-service.service`

mask if you want it prevented entirely

`sudo systemctl mask some-service.service`

2) Fix blocking network waits

- Common culprit: `network-online.target` or `NetworkManager-wait-online.service`. They often add several seconds to boot on embedded devices.
- Solution: don't require `network-online.target` for units that can start without network; use `Wants=` instead of `Requires=`; or remove `wait-online` service.

Example:

```
sudo systemctl disable NetworkManager-wait-online.service
```

3) Reduce udev and module load time (embedded-specific)

- Remove unused kernel modules (compile drivers into kernel instead of as modules for your board-specific build). Each module load adds io/udev activity.
- Prune udev rules that trigger slow scripts. For Yocto: ensure only necessary modules are in `IMAGE_INSTALL`.
- On BeagleBone, limit device-tree overlays or cape manager behavior that triggers many hotplug events.

4) Speed up storage I/O (often huge effect)

- Use a faster SD card or eMMC (class matters). I/O-bound boots on slow SD cards are common — I/O wait lines on your chart indicate that.
- Minimize files written/read at boot (move logs to tmpfs if acceptable).
- If using an initramfs that scans or fsck's partitions — reduce or disable fsck at boot (careful: only if safe).

5) Parallelize services (systemd)

- systemd is built for parallelization. Ensure your custom units do not incorrectly use `After=`/`Requires=` making them serialize unnecessarily.
- Use `Type=oneshot + RemainAfterExit=yes` carefully; prefer socket activation where possible.

6) Optimize initramfs / initrd

- If using a large initramfs that unpacks lots of files, try to shrink it or move non-essential work to later in boot.
- Build a minimal initramfs that only mounts the rootfs and starts systemd.

7) Kernel / bootloader tweaks

- Reduce U-Boot timeouts (e.g., `bootdelay`), remove unnecessary console prints. For BeagleBone, shorten MLO/bootloader waits if present.
- Keep DT overlays minimal; pre-apply necessary device tree at build time rather than loading overlays at boot.

8) Filesystem optimizations

- Mount frequently used directories using noatime (prevents writes on reads). In `/etc/fstab` add `noatime`.
- Use smaller/faster filesystems tuned for your storage medium.

9) Defer non-critical boot tasks

- For tasks not required at boot (updates, cronjobs, monitoring agents), defer them post-boot via `systemd` timers or at `boot-complete` target.

10) If you're building with Yocto

- Remove or reduce packages pulled into the image that add daemons or `udev` rules.
- Consider enabling faster init systems or cut down `systemd` units via image features.

How I would proceed next (if you want me to continue)

I can produce a *targeted* optimization plan and a ranked list of the slowest units if you provide any of the following now (pick one — no waiting needed, I'll analyze immediately):

1. Output of `systemd-analyze` and `systemd-analyze blame` (paste here).
2. `journalctl -b` for the boot shown in the chart (paste relevant ranges) — this helps identify timeouts.
3. Or re-export the bootchart SVG with visible numeric time ticks (or tell me the total boot time shown by `systemd-analyze`) so I can convert `px`→seconds and list the top offenders with exact seconds.

If you prefer, I'll start by giving you a prioritized checklist you can run on the board right now (I already included one above). If you run one command and paste the output, I'll convert it into specific changes to implement.

Quick checklist you can start with on the board (summary)

1. `systemd-analyze` → note total time.
2. `systemd-analyze blame | head -n 20` → identify top 20 slow services.
3. `systemctl disable --now <unneeded-service>` for non-essential services.
4. Check for `network-wait` services and disable if unnecessary.
5. Check `/etc/fstab` for `noatime`; consider mounting `var/log` on `tmpfs` if logs cause I/O.
6. Reduce `udev/module` count (rebuild kernel with drivers built-in where possible).
7. Replace slow SD card or use eMMC.

If you want, paste the output of `systemd-analyze + systemd-analyze blame` here and I'll map those results back to this SVG and give:

- exact seconds per service (converted from chart px if you provide total boot time),
- a prioritized “do this first” list of 6 concrete systemctl/kernel/Yocto edits, and
- a suggested Yocto image recipe tweak to permanently remove offending packages.

Shall I take one of those outputs (e.g., systemd-analyze blame) and map it to the chart now?

```
km@KernelMasters:/run/log$ systemd-analyze
```

Startup finished in 3.505s (kernel) + 24.882s (userspace) = 28.388s

graphical.target reached after 24.865s in userspace

```
km@KernelMasters:/run/log$ systemd-analyze blame
```

```
17.827s plymouth-quit-wait.service
11.791s vboxadd.service
4.884s snapd.service
4.523s dev-sda1.device
2.824s snap-snapd-24792.mount
2.795s snap-core18-2947.mount
2.602s snap-core18-2952.mount
2.538s snap-core24-1151.mount
2.446s snap-gnome\x2d3\x2d34\x2d1804-93.mount
2.366s snap-gnome\x2d46\x2d2404-125.mount
2.231s snap-gnome\x2dcharacters-805.mount
2.225s snap-gnome\x2dcharacters-761.mount
2.196s snap-gnome\x2dlogs-129.mount
2.123s snap-gtk\x2dcommon\x2dthemes-1535.mount
2.085s networkd-dispatcher.service
1.916s udisks2.service
1.761s snap-gnome\x2dcalculator-884.mount
1.644s ModemManager.service
1.629s dev-loop10.device
1.621s snap-mesa\x2d2404-912.mount
1.605s dev-loop8.device
```

1.598s dev-loop9.device
1.597s dev-loop5.device
1.405s snap-snapd-25202.mount
1.379s dev-loop1.device
1.370s dev-loop2.device
1.346s NetworkManager.service
1.310s dev-loop12.device
1.305s dev-loop11.device
1.272s dev-loop0.device
1.272s dev-loop6.device
1.270s snap-gnome\x2d3\x2d38\x2d2004-87.mount
1.266s dev-loop4.device
1.252s dev-loop3.device
1.176s dev-loop7.device

km@KernelMasters:/run/log\$ systemd-analyze critical-chain

The time after the unit is active or started is printed after the "@" character.

The time the unit takes to start is printed after the "+" character.

graphical.target @24.865s
└─multi-user.target @24.865s
└─vboxadd-service.service @17.382s +56ms
└─vboxadd.service @5.589s +11.791s
└─basic.target @5.112s
└─sockets.target @5.112s
└─snapd.socket @5.111s +779us
└─sysinit.target @5.094s
└─snapd.apparmor.service @4.283s +810ms
└─apparmor.service @3.701s +578ms

```

└─local-fs.target @3.700s
  └─run-user-121.mount @17.728s
    └─swap.target @1.013s
      └─swapfile.swap @598ms +398ms
        └─systemd-remount-fs.service @540ms +34ms
          └─systemd-journald.socket @523ms
            └─system.slice @523ms
              └─.slice @516ms

```

boot-time profiling data from systemd-analyze.

how to optimize your boot:

Total boot time = 28.388s

Kernel time = 3.505s

Userspace time = 24.882s

Graphical target reached after 24.865s

So, most delay is in userspace (≈ 25 s) — which means system services and startup daemons are taking time, not the kernel.

Top Slow Services (from systemd-analyze blame)

Service	Time	Cause	Solution
plymouth-quit-wait.service	17.827s	Waiting for splash screen to close	Disable Plymouth splash screen
vboxadd.service	11.791s	VirtualBox Guest Additions startup delay	Mask or disable if not required
snapd.service	4.884s	Snap daemon slow to initialize	Disable or remove Snap packages
snap-*mount	2–3s each	Each Snap mount adds delay	Replace snaps with native .deb packages if possible
networkd-dispatcher.service	2.085s	Network init delay	Acceptable, can tune NetworkManager

Service	Time	Cause	Solution
udisks2.service	1.916s	Disk management daemon	Acceptable, can be masked on servers without GUI

Optimization Steps

1 Disable Plymouth

This alone will save ~17 seconds.

```
sudo systemctl disable plymouth-quit-wait.service
```

```
sudo systemctl mask plymouth-quit-wait.service
```

```
sudo systemctl disable plymouth-start.service
```

And edit GRUB to remove splash:

```
sudo nano /etc/default/grub
```

Find:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
```

Change to:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet"
```

Then:

```
sudo update-grub
```

2 Disable VirtualBox Guest Additions (if not needed)

If this is a native system, not a VM:

```
sudo systemctl disable vboxadd.service vboxadd-service.service
```

```
sudo systemctl mask vboxadd.service vboxadd-service.service
```

If it is a VM, update Guest Additions:

```
sudo /sbin/rcvboxadd setup
```

3 Remove or Limit Snap

Snap adds multiple mount units and a background daemon.

To check which snaps are installed:

```
snap list
```

If you don't need Snap:

```
sudo systemctl disable snapd.service
```

```
sudo systemctl mask snapd.service
```

```
sudo apt purge snapd
```

This can cut 5–10 seconds.

4 Enable Parallel Startup

You can allow systemd to start services in parallel:

```
sudo systemctl edit systemd.conf
```

Add under [Manager]:

```
DefaultTimeoutStartSec=10s
```

```
DefaultTimeoutStopSec=10s
```

Then reload:

```
sudo systemctl daemon-reexec
```

```
sudo systemctl daemon-reload
```

5 Analyze Dependencies

You can visually inspect delays:

```
systemd-analyze plot > boot-analysis.svg
```

Then open the SVG file in a browser to see a timeline graph of every service.

Expected Result After Optimization

After disabling Plymouth, Snap, and VirtualBox services:

- Kernel: ~3s (unchanged)
- Userspace: ~8–10s
- Total Boot: ~11–13s